

Switching Controller Synthesis for Hybrid Systems Against STL Formulas



Han Su^{1,2} , Shenghua Feng³ , Sinong Zhan⁴ , and Naijun Zhan^{5,1}

¹ State Key Lab. of Computer Science, Institute of Software, Chinese Academy of Sciences, Beijing, China {suhan,znj}@ios.ac.cn

² University of Chinese Academy of Sciences, Beijing, China

³ Zhongguancun Laboratory, Beijing, China fengsh@zgclab.edu.cn

⁴ Department of Electrical and Computer Engineering, Northwestern University
SinongZhan2028@u.northwestern.edu

⁵ School of Computer Science, Peking University, Beijing, China

Abstract. Switching controllers play a pivotal role in directing hybrid systems (HSs) towards the desired objective, embodying a “correct-by-construction” approach to HS design. Identifying these objectives is thus crucial for the synthesis of effective switching controllers. While most of existing works focus on safety and liveness, few of them consider timing constraints. In this paper, we delve into the synthesis of switching controllers for HSs that meet system objectives given by a fragment of STL, which essentially corresponds to a reach-avoid problem with timing constraints. Our approach involves iteratively computing the state sets that can be driven to satisfy the reach-avoid specification with timing constraints. This technique supports to create switching controllers for both constant and non-constant HSs. We validate our method’s soundness, and confirm its relative completeness for a certain subclass of HSs. Experiment results affirm the efficacy of our approach.

Keywords: Hybrid Systems · Switching Controller Synthesis · Signal Temporal Logic · Reach-Avoid.

1 Introduction

Hybrid systems (HSs) provide a robust mathematical specification in modeling cyber-physical systems (CPS) with their unique fusion of continuous physical dynamics and discrete switching behaviors. Many CPSs are often complex and safety-critical which necessitates intricate control specifications. Switching controller synthesis offers a formal guarantee of the given specification of HS. Its

This work has been partially funded by the NSFC under grant No. 62192732 and 62032024, by the National Key R&D Program of China under grant No. 2022YFA1005101, by the CAS Project for Young Scientists in Basic Research under grant No. YSBR-040.

applications include attitude control in aerospace [3], aircraft collision-avoidance protocols in avionics [42], and pacemakers for treating bradycardia [52], etc.

With the escalating complexity of CPSs [43,44,53], the specifications required to ensure their proper functionality grow increasingly intricate. Among these, the importance of timing constraints becomes paramount [47,48]. This is evident in various scenarios, from orchestrating synchronized reactions in chemical processing [12] to ensuring seamless operations in multi-robot systems [23]. In this context, Signal Temporal Logic (STL), a rigorous formalism for defining linear-time properties of continuous signals [26], is exceptionally well-suited for specifying intricate timing constraints and qualitative properties of CPSs.

However, switching controller synthesis for HSs against STL specifications is not well addressed in the literature. The primary challenge arises from the complex interactions between continuous behaviors and discrete transitions. A common technique to synthesize switching controllers for HSs with complex specifications is the abstraction-based method [25,27]. This technique involves abstracting the continuous state space of each mode into a finite set of states, which often results in the loss of precise timing information for each mode. Consequently, the abstraction-based technique struggles with timing constraint analysis in the abstracted state space. In contrast, Mixed Integer Linear Programming (MILP) based technique [33] for switching controller synthesis against STL specification can provide precise timing information, but this method faces challenges in handling the intricate interactions of diverse discrete transitions between modes.

In this paper, we considered the switching controller synthesis problem for HSs against a fragment of STL specification, which essentially corresponds to a reach-avoid problem with timing constraints. To the best of our knowledge, this is the first work that uses STL to specify HSs with both discrete transitions and continuous dynamics. Similar work in [38] focused only on HSs with discrete time dynamics in each mode, significantly simplifying the problem. The key idea behind our approach involves iteratively computing a sequence of state-time sets (x, t) , state x and time t . These sets ensure that an HS, starting from state x at time t , adheres to the STL specification within a certain number (i.e., the number of iterations) of switches. The state-time sets are computed explicitly when the dynamics of the HSs are constant, and are inner-approximated when the dynamics are non-constant. Based on the state-time sets, we propose a sound and relatively complete method to synthesize a switching controller that satisfies the STL specification. Our experimental results demonstrate the efficacy of this approach.

The main contributions can be summarized as follows: (i) We conceptualize *state-time set* for HSs. (ii) We propose a methodology to synthesize switching controllers for HSs against a fragment of STL specification. (iii) We develop a prototype to demonstrate the efficiency and practical applicability of our methodology.

Organization. Sect. 2 gives an overview of our approach, Sect. 3 provides a recap of important preliminaries and formally defines the problem. We illustrate

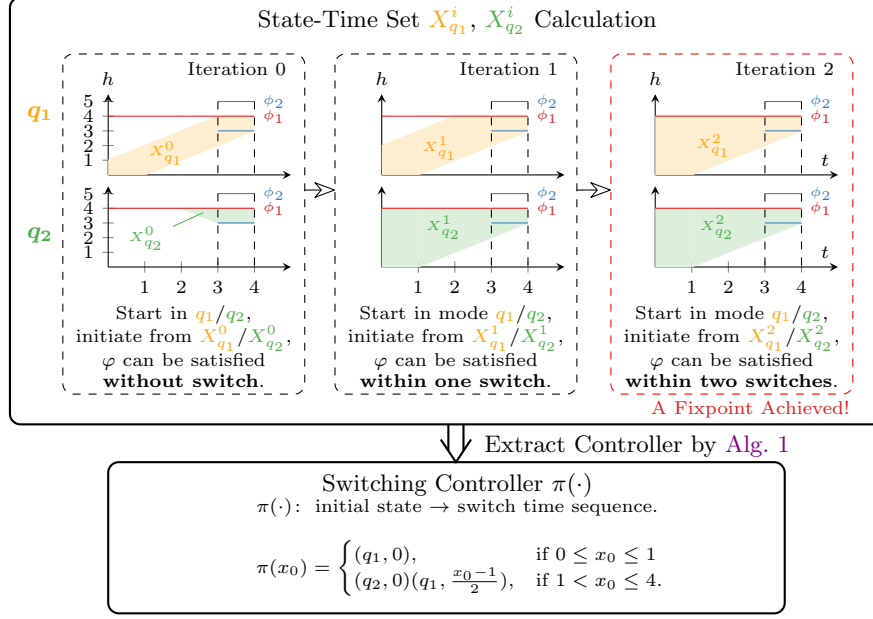


Fig. 1: An overview of our method

the calculation of the state-time sets in Sect. 4. Based on the state-time sets, Sect. 5 shows how to derive switching systems against a STL specification. In Sect. 6, we demonstrate the efficacy of our method through several examples. We discuss related work in Sect. 7 and draw conclusion in Sect. 8.

Due to space restrictions, proofs and benchmark details have been omitted, which can be found in an extended version of this paper [39].

2 An Illustrative Prelude

Example 1. In the reactor system depicted in Fig. 2, liquid is continuously consumed by the reaction and is replenished through pipe P . The system alternates between modes of adding liquid (q_1) and exclusively consuming it (q_2). The objectives are to keep the liquid level, h , between 0 and 4 meters, and to ensure that h remains between 3 and 5 meters at a certain point during a critical reaction phase - time interval 3 to 4, for proper interaction with the reactor rod R . These objectives can be given as an STL formula $\varphi = (0 \leq h \leq 4) \mathcal{U}_{[3,4]}(3 \leq h \leq 5)$. $\triangleleft$

We present the core idea behind our approach in Fig. 1. Initially, we compute the state-time set X_q^i iteratively. As shown in the upper block of Fig. 1, the set X_q^i encompasses states in mode q from which φ can be satisfied within i switches

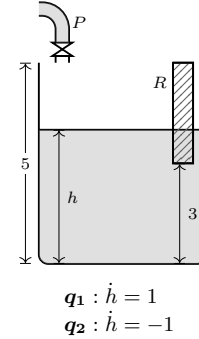


Fig. 2: Reactor System

(as detailed in [Sect. 5](#)). Once these *state-time sets* are determined, the switching controllers can be synthesized using the methods outlined in [Alg. 1](#) and [Alg. 2](#).

3 Notations and Problem Formulation

Notations. Let $\mathbb{N}$, $\mathbb{R}$, and $\mathbb{R}_{\geq 0}$ denote the set of natural, real, and non-negative real numbers, respectively. Given vector $x \in \mathbb{R}^n$, x_i refers to its i -th component, and $p[x = u]$ denotes the replacement of x by u for any predicate p where x serves as a variable.

Differential dynamics. We consider a class of dynamical systems featuring differential dynamics governed by ordinary differential equations (ODEs) of the form $\dot{\mathbf{x}} = f(\mathbf{x})$, where f is a continuous differentiable function. Given an initial state $x_0 \in \mathbb{R}^n$, there exist a unique solution $\mathbf{x} : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}^n$ in the sense that $\dot{\mathbf{x}}(t) = f(\mathbf{x}(t))$ for $t \geq 0$ and $\mathbf{x}(0) = x_0$.

Switched systems. A switched system is defined as a tuple $\Phi = (Q, F, \text{Init}, \pi)$, where

- $Q \triangleq \{q_1, q_2, \dots, q_m\}$ is a finite set of discrete modes.
- $F \triangleq \{f_q \mid q \in Q\}$ is a set of vector fields, and each mode $q \in Q$ endows with a unique vector field f_q which specifies how system evolves in mode q .
- $\text{Init} \subseteq \mathbb{R}^n$ is a set of initial states.
- $\pi : \text{Init} \rightarrow (\mathbb{R}_{\geq 0} \rightarrow Q)$ is a switching controller. The controller maps each initial state $x_0 \in \text{Init}$ to a piecewise constant function $\pi(x_0)$, which in turn maps a time t to the corresponding control mode $\pi(x_0)(t)$.

Given any initial state x_0 , the dynamics of the switched system Φ is governed by equation $\dot{\mathbf{x}}(t) = f_{\pi(x_0)(t)}(\mathbf{x}(t))$ with initial condition $\mathbf{x}(0) = x_0$.

Signal temporal reach-avoid. We consider a fragment of signal temporal logic, namely *signal temporal reach-avoid* formula (ST-RA for short). The syntax of ST-RA is defined by

$$\begin{aligned}\phi &::= \mu(x, t) \geq 0 \mid \neg\phi \mid \phi \wedge \phi \\ \varphi &::= \phi_1 \mathcal{U}_I \phi_2\end{aligned}$$

where ϕ is a Boolean combination of predicates over x and time t , $I \triangleq [l, u]$ is a closed time interval for some $0 \leq l \leq u$. Intuitively, an ST-RA formula φ expresses the requirement that the system should reach ϕ_2 while avoid leaving ϕ_1 within time frame I . The semantics of ST-RA formula, in alignment with STL, is defined as the satisfaction of a formula φ with respect to a signal $\mathbf{x}$ and a time instant t .

Remark 1. Compared with the standard signal temporal logic (STL) [26], ST-RA formula does not allow nested “until” operator, this makes ST-RA formula a fragment of STL.

Formally, given function $\mathbf{x}: \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}^n$ (termed signal) and time τ , the satisfaction of φ at $(\mathbf{x}, \tau)$, denoted by $(\mathbf{x}, \tau) \models \varphi$, is inductively defined as follows:

$$\begin{aligned}
(\mathbf{x}, \tau) \models \mu(x, t) \geq 0 & \quad \text{iff} \quad \mu(\mathbf{x}(\tau), \tau) \geq 0; \\
(\mathbf{x}, \tau) \models \neg \phi & \quad \text{iff} \quad (\mathbf{x}, \tau) \not\models \phi; \\
(\mathbf{x}, \tau) \models \phi_1 \wedge \phi_2 & \quad \text{iff} \quad (\mathbf{x}, \tau) \models \phi_1 \text{ and } (\mathbf{x}, \tau) \models \phi_2; \\
(\mathbf{x}, \tau) \models \phi_1 \mathcal{U}_I \phi_2 & \quad \text{iff} \quad \exists \tau' \geq \tau, \text{ such that } \tau' - \tau \in I, (\mathbf{x}, \tau') \models \phi_2, \\
& \quad \text{and } \forall \tau'' \in [\tau, \tau'], (\mathbf{x}, \tau'') \models \phi_1.
\end{aligned}$$

Intuitively, the subscript I in the until operator $\mathcal{U}_I$ defines the timing constraints under which a signal must reach ϕ_2 while avoid leaving ϕ_1 .

Given a ST-RA formula ϕ , we say a switched system $\Phi = (Q, F, \text{Init}, \pi)$ models φ , denoted by $\Phi \models \varphi$, if $(\mathbf{x}, 0) \models \varphi$ for any trajectory $\mathbf{x}$ starting from initial set Init . We now formulate the problem addressed in this paper.

Problem Formulation. Suppose there exists a finite set of control modes $Q = \{q_1, q_2, \dots, q_m\}$ and associated vector fields $F = \{f_{q_1}, f_{q_2}, \dots, f_{q_m}\}$. Each mode $q \in Q$ is associated with a vector field f_q that governs the system's behavior in mode q . Let $\varphi = \phi_1 \mathcal{U}_I \phi_2$ be an ST-RA formula, a natural question is how to design a system that incorporates mode $q \in Q$ as subsystems, while ensuring any trajectory of the system satisfies φ . To address this, we formulate the problem as follows.

Synthesis of Switched System. Given a finite set of discrete modes $Q = \{q_1, q_2, \dots, q_m\}$, a set of vector fields $F = \{f_{q_1}, f_{q_2}, \dots, f_{q_m}\}$, and a ST-RA formula $\varphi = \phi_1 \mathcal{U}_I \phi_2$, the switched system synthesis problem aims to synthesize a switched system $\Phi = (Q, F, \text{Init}, \pi)$, such that $\Phi \models \varphi$.

Remark 2. The solutions to the above synthesis problem is inherently non-unique and may encompass trivialities, such as the one only with an empty initial set. Therefore, our goal is to identify a system with a nontrivial initial set Init .

4 State-Time Set and Its Calculation

This section dedicates to synthesize a switched system Φ that satisfies the given ST-RA formula $\varphi = \phi_1 \mathcal{U}_I \phi_2$. The key idea behind our approach is to compute a sequence of state-time sets $\{X_q^i\}_{q \in Q}$ for $i \in \mathbb{N}$, where X_q^i denotes the set of all (x, τ) such that starting from x at time τ in mode q , the system can be driven to reach ϕ_2 while satisfying ϕ_1 *within i times of switches*. In what follows, we first formally propose the concept of state-time sets and show how to calculate it explicitly. Subsequently, leveraging these state-time sets, we demonstrate the synthesis of a switched system that satisfies φ in [Sect. 5](#).

4.1 State-Time Sets

The concept of state-time set is formally summarised by the following definition.

Definition 1 (State-time sets). For any $i \in \mathbb{N}$ and any $q \in Q$, let X_q^i denote the set of all state-time pairs (x, τ) such that there exists a controller $\pi(x): [\tau, \infty) \rightarrow Q$, satisfying

- (i) $\pi(x)(\tau) = q$, and the piecewise constant function $\pi(x)$ contains at most i discontinuous points;
- (ii) $(x, \tau) \models \phi_1 \mathcal{U}_{I \div \tau} \phi_2$, where $\mathbf{x}$ is the solution of ODE $\dot{\mathbf{x}}(t) = f_{\pi(x)(t)}(\mathbf{x}(t), t)$ over $[\tau, \infty)$ with $\mathbf{x}(\tau) = x$, and $I \div \tau \triangleq [l - \tau, u - \tau] \cap \mathbb{R}_{\geq 0}$ for any interval $I = [l, u]$.

Intuitively, condition (ii) suggests that the system can be driven to reach ϕ_2 while satisfying ϕ_1 from x at time τ , and condition (i) indicates that the system initially remains in mode q , and the switching controller undergoes no more than i switches. From the above definition of state-time sets, the following results can be derived:

Corollary 1. The following properties hold for the state-time sets $\{X_q^i\}_{q \in Q}$:

1. For any $q \in Q$, $\{X_q^i\}$ is monotonically increasing, i.e. $X_q^0 \subseteq X_q^1 \subseteq X_q^2 \subseteq \dots$.
2. For any $i \in \mathbb{N}$ and any $x \in X_q^i[t=0]$, x can be driven to satisfy $\phi_1 \mathcal{U}_I \phi_2$, i.e. there exists a switching controller π , such that $(x, 0) \models \phi_1 \mathcal{U}_I \phi_2$, where $\mathbf{x}$ is the trajectory starting from x at time 0 under controller π , and $X_q^i[t=0] \triangleq \{x \mid (x, 0) \in X_q^i\}$ is the projection of X_q^i into $t = 0$.
3. $\cup_{i \in \mathbb{N}} \cup_{q \in Q} X_q^i[t=0]$ is the set of all states that can be driven to satisfy $\phi_1 \mathcal{U}_I \phi_2$.

According to Cor. 1, the state-time sets encompass the initial set of the switched system that we intend to synthesize. However, the state-time set and controller defined in Def. 1 are not given explicitly. To address this, we first elucidate the process of calculating the state-time sets.

The subsequent result establishes a relationship between the sets $\{X_q^i\}_{q \in Q}$ and $\{X_q^{i-1}\}_{q \in Q}$, forming the foundation for the inductive computation of state-time sets.

Theorem 1. Follow the notations as before, we have⁶

1. Given any $q \in Q$, $(x, \tau) \in X_q^0$ if and only if

$$(\mathbf{x}, \tau) \models \phi_1 \mathcal{U}(\phi_2 \wedge (t \in I)) \quad (1)$$

where $\mathbf{x}$ is the solution of ODE $\dot{\mathbf{x}}(t) = f_q(\mathbf{x}(t), t)$ over $[\tau, \infty)$ with $\mathbf{x}(\tau) = x$.

⁶ For any $a, b \in \mathbb{R}_{>0}$ such that $a \leq b$, the constraint $a \leq t \leq b$ is concisely denoted as $t \in [a, b]$.

2. Given any $q \in Q$, for any $i \geq 1$, $(x, \tau) \in X_q^i$ if and only if

$$\exists q' \neq q \in Q, (x, \tau) \models \phi_1 \mathcal{U} X_{q'}^{i-1} \quad (2)$$

where x is the solution of ODE $\dot{x}(t) = f_q(x(t), t)$ over $[\tau, \infty)$ with $x(\tau) = x$.

For any formula $\psi(u, v)$, let $\mathbf{QE}(\exists u, \psi(u, v)) \triangleq \{v \mid \exists u, \text{ s.t. } \psi(u, v) \text{ holds}\}$ denote the set of all v for which $\exists u, \psi(u, v)$ is true. Utilizing this notation, the state-time sets can be represented inductively.

Theorem 2. For any $q \in Q$, suppose the solution of ODE $\dot{x}(t) = f_q(x(t))$ with initial x at time τ is denoted by $\Psi(\cdot; x, \tau, q)$, then the state-time sets can be inductively represented by

$$X_q^0 = \mathbf{QE} \left(\exists \delta \geq 0, \left(\phi_2[(x, t) = (\Psi(t + \delta; x, t, q), t + \delta)] \wedge (t + \delta \in I) \right) \right. \\ \left. \wedge \left(\forall 0 \leq h \leq \delta, \phi_1[(x, t) = (\Psi(t + h; x, t, q), t + h)] \right) \right) \quad (3)$$

$$X_q^i = \bigvee_{q' \neq q} \mathbf{QE} \left(\exists \delta \geq 0, \left(X_{q'}^{i-1}[(x, t) = (\Psi(t + \delta; x, t, q), t + \delta)] \right) \right. \\ \left. \wedge \left(\forall 0 \leq h \leq \delta, \phi_1[(x, t) = (\Psi(t + h; x, t, q), t + h)] \right) \right) \quad (4)$$

for any $q \in Q$ and any $i \in \mathbb{N}$.

Remark 3. When $\psi(u, v)$ consists of a Boolean combination of polynomial inequalities, a decidable procedure, such as cylindrical algebraic decomposition [2], exists for computing $\mathbf{QE}(\exists u, \psi(u, v))$. This procedure exhibits a complexity that is double exponential with respect to the number of variables involved.

Remark 4. Our methodology essentially shares the idea of backward induction in controller synthesis for timed games [9]. However, our approach diverges in two key aspects: (1) the safety/target sets and timing constraints are intricately interwoven in ST-RA formula, necessitating their concurrent consideration at each step of the induction process; (2) our method operates within an infinite-dimensional space due to the continuous nature of the state space, in contrast to the backward induction for timed games, which is confined to a finite set of k-polyhedra.

4.2 Computing/Approximating State-Time Sets

Although [Thm. 2](#) offers an inductive representation of X_q^i , the explicit computation of [Eqs. \(3\) and \(4\)](#) are challenging. This difficulty arises from two main factors: (i) the necessity to explicitly solve the ordinary differential equation in each mode, and (ii) the high complexity of $\mathbf{QE}$, and the potential inclusion of non-elementary functions (such as exponential functions) in [Eqs. \(3\) and \(4\)](#), for which a generally decidable procedure to solve $\mathbf{QE}$ may not exist.

To address the difficulties outlined above, we categorize the dynamics into constant and non-constant systems. For the constant dynamics, its solution can be directly computed, and there exists a decidable procedure to solve QE with a complexity polynomially dependent on the formula length. For the non-constant dynamics, due to their high complexity, we forego an explicit solution for the state-time set and instead demonstrate a method to approximate this set.

Constant dynamics. Suppose the dynamics within each mode $q \in Q$ is constant, and both ϕ_1 and ϕ_2 are Boolean combinations of linear inequalities, Eqs. (3) and (4) can be effectively solved using readily available solvers, such as Z3 [10]. Thm. 2 directly implies the following result.

Corollary 2. *Following the notations as before, suppose the dynamics within each mode is constant, i.e. $f_q = a_q$ for any $q \in Q$, and both ϕ_1 and ϕ_2 are Boolean combinations of linear inequalities, then $\{X_q^i\}_{q \in Q}$ can be inductively solved by Eqs. (3) and (4) with $\Psi(t; x, \tau, q) = x + (t - \tau) \cdot a_q$.*

Remark 5. Although QE on polynomial constraints is double-exponential in general [2], constant dynamics facilitate a relatively efficient (*polynomial in formula length*) solving procedure. This comes from the following observation: (1) the QE procedure in Eqs. (3) and (4) operates in polynomial time when the constraints are linear and involve only a single existential and a single universal variable [45, Thm 6.2]. (2) if X_q^{i-1} is linear for all $q \in Q$, then X_q^i is also linear.

We now illustrate the computation process of X_q^i via the following example.

Example 2. Let's reconsider the reactor system in Exmp. 1. The reactor system consists of two modes q_1 and q_2 with $f_{q_1} = 1$, $f_{q_2} = -1$, and the liquid level requirement is $\varphi = (0 \leq h \leq 4) \mathcal{U}_{[3,4]}(3 \leq h \leq 5)$.

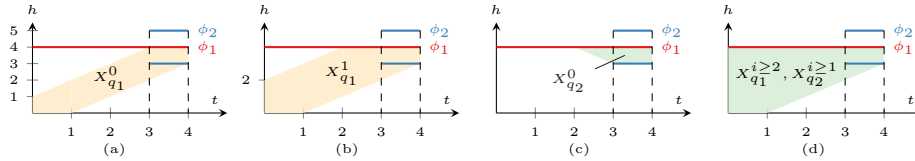


Fig. 3: The state-time sets calculation of the reactor system in Exmp. 1. The state-time sets reach a fixpoint after 2 iteration.

Based on Eqs. (3) and (4), the state-time sets we calculate are illustrated in Fig. 3. The procedure reaches a fixpoint within 2 iterations for any $q \in Q$. $\triangleleft$

Non-constant dynamics. Assuming that the dynamics are non-constant, the exact computation of Eqs. (3) and (4) may prove to be overly complex or potentially undecidable. We thus seek to *inner-approximate* the state-time sets. According to Thm. 1,

- X_q^0 is the set from which the system in mode q will satisfy $\phi_1 \mathcal{U}(\phi_2 \wedge (t \in I))$;
- X_q^i is the set from which the system in mode q will satisfy $\phi_1 \mathcal{U} X_{q'}^{i-1}$ for some $q' \in Q$.

We identify that the crucial element for inner-approximating the state-time sets lies in employing a method that finds sets from which the system will satisfy a classical ‘until’ or ‘reach-avoid’ formula⁷. Numerous studies have explored this issue; in this paper, we employ the approach proposed in [51].

Theorem 3 (Inner-approximation of Reach-avoid Set [51]). *Given dynamic system $\dot{x}(t) = f(x(t))$, safety set $\psi_1 \subseteq \mathbb{R}^n$ and target set $\psi_2 \subseteq \mathbb{R}^n$. If there exists continuously differentiable function $v(x) : \overline{\psi_1} \rightarrow \mathbb{R}$ and $w(x) : \overline{\psi_1} \rightarrow \mathbb{R}$, satisfying⁸*

$$\begin{cases} \nabla_x v(x) \cdot f(x) \geq 0, & \forall x \in \overline{\psi_1 \setminus \psi_2}, \\ v(x) - \nabla_x w(x) \cdot f(x) \leq 0, & \forall x \in \overline{\psi_1 \setminus \psi_2}, \\ v(x) \leq 0, & \forall x \in \partial\psi_1, \end{cases}$$

then any trajectory starting from $\{x \mid v(x) \geq 0\}$ satisfies formula $\psi_1 \mathcal{U} \psi_2$.

Remark 6. The synthesis of function $v(x)$ and $w(x)$ can be reduce to a SDP problem. For a detailed formulation, we refer the reader to [51].

Since the state-time sets X_q^i depend on both the state x and time t , we first lift the dynamics of each mode to a higher dimension that incorporates time t . Specifically, the dynamics in mode q are transformed into $(\dot{x}, \dot{t}) = (f_q, 1)$. Subsequently, employing [Thm. 3](#) and [Thm. 1](#), we can inductively inner-approximate the state-time set X_q^i . The resulting approximation is denoted by $\tilde{X}_q^i$.

Example 3. Consider a temperature control system featuring two modes, q_1 and q_2 , with dynamics given by $f_{q_1} = 20 - 0.2x - 0.001x^2$ and $f_{q_2} = -0.2x - 0.001x^2$, where x represents the temperature. The control objective is defined by the ST-RA formula $\varphi = (20 \leq x \leq 80) \mathcal{U}_{[4,5]} (60 \leq x \leq 80)$.

[Fig. 4](#) presents the result obtained by inner-approximating⁹ $X_{q_1}^0$, $X_{q_2}^0$, $X_{q_1}^1$, and $X_{q_2}^1$. Based on the results, we observe that when x is within the range of $[20, 80]$ in mode q_1 , the system can satisfy φ without any switching. However, for $x \in [20, 80]$ in mode q_2 , at least one switch is necessary for φ to be satisfied.

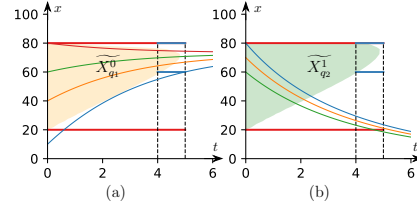


Fig. 4: The state-time sets approximation of temperature controller system

⁷ This problem is also referred to as the inner approximation of the reach-avoid problem

⁸ $\nabla_x v(x)$ represents the gradient of $v(x)$ with respect to x , $\overline{\psi_1}$ denotes the closure of set ψ_1 and $\partial\psi_1$ refers to the boundary of ψ_1 .

⁹ The approximation of $X_{q_2}^0$ and $X_{q_1}^1$ is an empty set, hence it is not depicted.

Algorithm 1 Synthesis of Switched system

Require: $Q, F, \varphi = \phi_1 \mathcal{U}_I \phi_2$, and $k \triangleright k$ is the upper bound of switching time
Ensure: A switched system $\Phi = (Q, F, \text{Init}, \pi)$, such that $\Phi \models \varphi$

```

1: for all  $q \in Q$  do
2:    $X_q^0 \leftarrow$  inner-approximate/explicitly calculate  $X_q^0$ 
3:    $\text{Init}(q)^0 \leftarrow X_q^0[t=0]$ 
4: end for
5: for  $i = 1, 2, \dots, k$  do
6:   for all  $q \in Q$  do
7:      $X_q^i \leftarrow$  inner-approximate/explicitly calculate  $X_q^i$ 
8:      $\text{Init}(q)^i \leftarrow (X_q^i \setminus X_q^{i-1})[t=0] \triangleright \text{Init}(q)^i$  is recorded for controller synthesis
9:   end for
10: end for
11:  $\text{Init} \leftarrow \cup_{q \in Q} \cup_{i=0}^k \text{Init}(q)^i \triangleright$  Initial set
12: Call Alg. 2 to obtain controller  $\pi \triangleright$  Given any  $x_0 \in \text{Init}$ , Alg. 2 computes
the controller that drives  $x_0$  to satisfy  $\varphi$ 

```

5 Synthesizing Switched Systems

In this section, we demonstrate the synthesis of a switched system Φ that conforms to the formula $\varphi = \phi_1, \mathcal{U}_I, \phi_2$. This synthesis builds on the state-time sets introduced in Section 4. We initially outline the synthesis procedure for the switched system in [Alg. 1](#) and subsequently describe the extraction of a switching controller in [Alg. 2](#).

Switched System Synthesis. We now summary the synthesis algorithm in [Alg. 1](#). Given any $k \in \mathbb{N}$ that serves as a prescribed upper bound of switching time, [Alg. 1](#) inductively calculates/inner-approximates¹⁰ state-time sets $\{X_q^i\}_{q \in Q}$ (line 2, 7), and partition $X_q^k[t=0]$ into $\text{Init}(q)^i \triangleq (X_q^i \setminus X_q^{i-1})[t=0]$ (line 3, 8) for $i = 0, 1, \dots, k$. $\text{Init}(q)^i$ denote the set of states (in mode q) that can be driven to satisfy φ with *at least i times* of switching (cf. [Cor. 1](#)). The initial set is defined by

$$\text{Init} = \cup_{q \in Q} \cup_{i=0}^k \text{Init}(q)^i,$$

which contains states that can be driven to satisfy φ within k times of switching, and the switching controller π is synthesized by [Alg. 2](#) (line 12).

Switching controller synthesis. For any $x_0 \in \text{Init}$, [Alg. 2](#) computes the controller that drives x_0 to satisfy φ . [Alg. 2](#) first finds $\text{Init}(q_0)^l$ that contains x_0 with l be the smallest index (line 1). l is the smallest switching time that can drive x_0 to satisfy φ , and the subscript q_0 indicates x_0 first lies in mode q_0 .

Line 4–10 find the next switching time and switching mode. Let $\text{Reach}(t; x_0, t_0, q)$ denote the over-approximation of the reachable set starting from (x_0, t_0) in mode

¹⁰ To clarify, we continue to use X_q^i to represent the inner approximation of the state-time sets, rather than using $\widetilde{X}_q^i$.

Algorithm 2 Switching controller synthesis

Require: $x_0, \{X_q^i\}_{i=0}^k$, and $\{\text{Init}(q)^i\}_{i=0}^k$ $\triangleright x_0$ is the initial state
Ensure: $\pi(x_0)$ $\triangleright$ The switching controller

- 1: Find the initial set $\text{Init}(q_0)^l$ that includes x_0 and has the smallest index l
- 2: Select q_0 as initial mode, $t_0 \leftarrow 0$
- 3: **for** $j = 1, \dots, l$ **do**
- 4: **for** $q \in Q$ **do**
- 5: **if** $\text{Reach}(\tilde{t}; x_{j-1}, t_{j-1}, q_{i-1}) \subseteq X_{\tilde{q}}^{l-j}[t = \tilde{t}]$ for some $\tilde{t} > t_{j-1}, \tilde{q} \in Q$ **then**
- 6: Select $t_j \leftarrow \tilde{t}, q_j \leftarrow \tilde{q}$
- 7: $x_j \leftarrow \text{Reach}(t_j; x_{j-1}, t_{j-1}, q_{j-1})$
- 8: **Break**
- 9: **end if**
- 10: **end for**
- 11: **end for**
- 12: $\pi(x_0) = (q_0, t_0)(q_1, t_1) \dots (q_l, t_l)$ $\triangleright$ Representing a piecewise constant function
such that $\pi(x_0)(t) = q_i$ if $t_i \leq t < t_{i+1}$

q at time t . Next switching time $\tilde{t}$ and switching mode $\tilde{q}$ are chosen to ensure that the system enters $X_{\tilde{q}}^{l-j}$ at time $\tilde{t}$ in mode q_{j-1} , this is formally encoded by

$$\text{Reach}(\tilde{t}; x_{j-1}, t_{j-1}, q_{i-1}) \subseteq X_{\tilde{q}}^{l-j}[t = \tilde{t}].$$

In line 12, the controller π maps x_0 to a piecewise constant function $\pi(x_0) = (q_0, t_0)(q_1, t_1) \dots (q_l, t_l)$, which represents a function that maps t to q_i if $t_i \leq t < t_{i+1}$.

Remark 7. Numerous methods are available to estimate the reachable set of a dynamic system [6,49,50]. In this paper, we employ Flow* [7], a method based on Taylor model, to over-approximate the reachable set.

Remark 8. Assuming that the dynamics (i.e. f_q) within each mode remain constant, the reachable set can be explicitly calculated. This, in conjunction with the explicit calculation of state-time sets, is crucial for demonstrating relative completeness in the context of constant dynamics (c.f. Thm. 4).

Remark 9. For non-constant dynamics, since the state-time sets and reachable sets are inner- and over-approximated, there may exist an initial state x_0 that can be driven to satisfy the ST-RA formula, while our method fails to identify a controller.

We now illustrate our approach through two examples.

Example 4. In Exmp. 2, we have obtained the state-time sets $\{X_{q_1}^i, X_{q_2}^i\}$ for $i \leq 2$, thus, according to Alg. 1 (with $k = 2$), we have

$$\begin{aligned} \text{Init}(q_1)^0 &= [0, 1], & \text{Init}(q_1)^1 &= (1, 2], & \text{Init}(q_1)^2 &= (2, 4] \\ \text{Init}(q_2)^0 &= \emptyset, & \text{Init}(q_2)^1 &= [0, 4], & \text{Init}(q_2)^2 &= \emptyset. \end{aligned}$$

Based on these sets, we can synthesize a switched system Φ with $\text{Init} = \{h \mid 0 \leq h \leq 4\}$. The corresponding switching controller π is defined by

$$\pi(x_0) = \begin{cases} (q_1, 0), & \text{if } 0 \leq x_0 \leq 1 \\ (q_2, 0)(q_1, \frac{x_0-1}{2}), & \text{if } 1 < x_0 \leq 4. \end{cases} \quad \triangleleft$$

Example 5. Let's reconsider [Exmp. 3](#), we demonstrate our approach by synthesizing the switching controller for initial state $x_0 = 80$ in mode q_2 . The reachable set $\text{Reach}(t; x_0, t_0, q_2)$ is represented by green boxes in [Fig. 5](#). We observe the reachable set will enter $X_{q_1}^0$ for any $t \in [0, 2]$, this implies initial state $x_0 = 80$ in mode q_2 can be driven to satisfy φ if the system switches into mode q_1 within time interval $[0, 2]$, i.e. $\pi(80) = (q_2, 0)(q_1, \tilde{t})$ for any $\tilde{t} \in [0, 2]$. $\triangleleft$

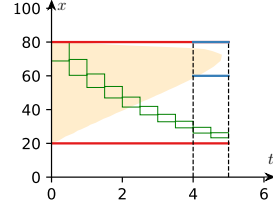


Fig. 5: Switching controller synthesis of [Exmp. 3](#)

The following result states the advantages of our approach.

Theorem 4 (Soundness, Relative Completeness, Minimal Switching Property). *Given modes Q , vector fields F , and formula $\varphi = \phi_1 \mathcal{U}_I \phi_2$, the following results hold:*

1. *Alg. 1 is sound, that is $\Phi \models \varphi$;*
2. *Alg. 1 is relatively complete for constant dynamics: for any $x \in \mathbb{R}^n$, if x can be driven to satisfy φ with some controller π , then there exists $k \in \mathbb{N}$ ¹¹, such that the initial set of the synthesized switched system contains x .*
3. *The controller synthesized in Alg. 2 features minimal switching property for constant dynamics: for any $x_0 \in \text{Init}$, there does not exist any controller π' , that can drive x_0 to satisfy φ with switching time (equivalently, number of discontinuous points of $\pi'(x_0)$) less than $\pi(x_0)$.*

Remark 10. Suppose the dynamic in each mode can be explicitly solved and there exists a decidable procedure for solving $\text{QE}(\cdot)$ in [Eqs. \(3\) and \(4\)](#), then [Alg. 1](#) is also relatively complete and the corresponding controller also features minimal switching property.

6 Experimental Evaluation

We develop a prototype¹² of our synthesis method in Python, employing the Z3 solver [\[10\]](#) to explicitly compute the state-time sets for HSs with constant dynamics. For HSs with linear or polynomial dynamics, we use the semidefinite programming solver MOSEK [\[29\]](#) to approximate the state-time set. The prototype is evaluated on various benchmark examples using a laptop with a 3.49GHz Apple M2 processor, 8GB RAM, and macOS 14.3.

Table 1: ST-RA Specifications

Model	ST-RA Formulas
Reactor [54]	$\varphi : (10 \leq \text{tempe} \leq 90) \wedge (0 \leq \text{cooling} \leq 1) \mathcal{U}_{[15,20]} (40 \leq \text{tempe} \leq 50)$
WaterTank [32]	$\varphi_1 : (10 \leq \text{lev}_0 \leq 95) \wedge (10 \leq \text{lev}_1 \leq 95) \wedge (\text{lev}_0 - \text{lev}_1 \leq 10) \mathcal{U}_{[50,60]} (50 \leq \text{lev}_0 \leq 80) \wedge (50 \leq \text{lev}_1 \leq 80)$
	$\varphi_2 : (10 \leq \text{lev}_0 \leq 95) \wedge (10 \leq \text{lev}_1 \leq 95) \wedge (\text{lev}_0 - \text{lev}_1 \leq 10) \mathcal{U}_{[30,40]} (50 \leq \text{lev}_0 \leq 80) \wedge (50 \leq \text{lev}_1 \leq 80)$
	$\varphi_3 : (10 \leq \text{lev}_0 \leq 95) \wedge (10 \leq \text{lev}_1 \leq 95) \mathcal{U}_{[30,40]} (50 \leq \text{lev}_0 \leq 80) \wedge (50 \leq \text{lev}_1 \leq 80)$
CarSeq [5]	$\varphi_1 : (1 \leq \text{pos}_0 - \text{pos}_1 \leq 3) \mathcal{U}_{[2,3]} (20 \leq \text{pos}_0 \leq 25)$
	$\varphi_2 : (1 \leq \text{pos}_0 - \text{pos}_1 \leq 3) \wedge (1 \leq \text{pos}_1 - \text{pos}_2) \mathcal{U}_{[2,3]} (20 \leq \text{pos}_0 \leq 25)$
	$\varphi_3 : (1 \leq \text{pos}_0 - \text{pos}_1 \leq 3) \wedge (1 \leq \text{pos}_1 - \text{pos}_2 \leq 3) \wedge (1 \leq \text{pos}_2 - \text{pos}_3) \mathcal{U}_{[2,3]} (20 \leq \text{pos}_0 \leq 25)$
Oscillator [51]	$\varphi : (x^2 + y^2 \leq 1) \mathcal{U}_{[3,4]} (x^2 + y^2 \leq 0.01)$
Temperature [5]	$\varphi_1 : \wedge_{i=1,2,3} (23 \leq \text{temp}_i \leq 29) \mathcal{U}_{[8,10]} \wedge_{i=1,2,3} (26 \leq \text{temp}_i \leq 28)$
	$\varphi_2 : \wedge_{i=1,2,3} (23 \leq \text{temp}_i \leq 29) \mathcal{U}_{[8,10]} \wedge_{i=1,2,3} (26 \leq \text{temp}_i \leq 28) \wedge (\text{temp}_2 \leq \text{temp}_1)$
	$\varphi_3 : \wedge_{i=1,2,3} (23 \leq \text{temp}_i \leq 29) \mathcal{U}_{[8,10]} \wedge_{i=1,2,3} (26 \leq \text{temp}_i \leq 28) \wedge (\text{temp}_2 \leq \text{temp}_1) \wedge (\text{temp}_3 \leq \text{temp}_2)$

More detail explanation of the ST-RA formula can be found in the full version of this paper [39].

As shown in Table 1, our experiments involve five distinct models, with three exhibiting constant dynamics and two exhibiting non-constant dynamics. We adjust the model scale or the ST-RA formula for each model to assess the efficacy of our method under varying conditions. And there are 15 different benchmarks in total included in our study. Table 2 details the empirical results of the benchmarks. In each case, the synthesis process continues iterating until either a fixpoint is achieved or the maximum calculation time of 5 minutes is met.

Our empirical results illustrate that our method is capable of effectively synthesizing controllers for models with both constant and non-constant dynamics. Notably, for models with constant dynamics, the iterative process tends to converge to a fixpoint, meaning that a complete controller is achieved. Moreover, the synthesis time for these controllers is significantly influenced by both the scale of the model and the complexity of ST-RA formulas. Specially, our analysis reveals: (i) an increased number of modes (Reactor) or a higher state dimension (CarSeq) both lead to prolonged synthesis times, (ii) more intricate predicates or larger future-reach time¹³ (WaterTank) results in increased synthesis times. For the third benchmark within CarSeq, the model does not reach a fixpoint, primarily because the large model scale rapidly increase the formula size, posing substantial challenges for the Z3 solver.

When dealing with non-constant dynamics, an approximation method is applied, thereby a fixpoint might not be achievable. The influence of model scale on

¹¹ In fact, k can be chosen to the number of discontinuous points of $\pi(x)$.

¹² Available at https://github.com/Han-SU/BenchMark_STLControlSyn4HS

¹³ Future-reach time refers to the maximum time horizon required to verify the correctness of an STL formula [5], in WaterTank, the future-reach times of φ_1 , φ_2 , and φ_3 are 60, 40, and 40 respectively.

Table 2: Empirical results on benchmark examples

Model	Dynamics	ST-RA	Model Scale		Synthesis Time	
			n_{dim}	n_{mode}	#Iter.	Time (s)
Reactor [54]	Const	φ	2	4	6 (fp)	0.31
		φ	2	8	6 (fp)	4.14
		φ	2	10	6 (fp)	8.01
WaterTank [32]	Const	φ_1	2	7	9 (fp)	18.04
		φ_2	2	7	6 (fp)	10.63
		φ_3	2	7	6 (fp)	5.24
CarSeq [5]	Const	φ_1	2	4	5 (fp)	1.12
		φ_2	3	8	7 (fp)	47.41
		φ_3	4	16	4	134.79
Oscillator [51]	Poly	φ	2	3	6	77.20
		φ	2	4	6	106.09
		φ	2	5	6	155.77
Temperature [5]	Linear	φ_1	3	8	5	236.99
		φ_2	3	8	5	293.66
		φ_3	3	8	5	252.32

Dynamics: the type of continuous dynamics; ST-RA: formulas to be satisfied (cf. Table 1); n_{dim} : dimension of state; n_{mode} : number of modes; #Iter.: number of iterations, (fp) means the synthesized set X_q^i (cf. Sect. 5) reach a fixpoint at current iteration.

synthesis time remains consistent with that observed in constant ODE models, as evidenced in *Oscillator*. Interestingly, the synthesis time for controllers using approximation methods is less affected by the complexity of the ST-RA formula. For example, in *Temperature*, despite φ_3 being more complex than φ_2 , it requires less synthesis time, primarily because the complexity of SDP is influenced more by state space dimensions than by constraints.

Overall, our method exhibits a high capability in synthesizing switching controller for HSs with various dynamics. It can achieve sound and complete results for constant dynamics within a reasonable time. For more general dynamics, our method can still synthesize a sound result in a reasonable time.

7 Related Work

HSs have been a key research focus in the academic community [46]. The autonomous *verification* and *synthesis* of HSs began from timed automata [1]. Subsequently, various mathematical models, including hybrid automata [17, 18], delay differential systems [14, 55], stochastic systems [30, 13], have been employed to reason about HSs. For a survey of these methods, we refer to [11].

In the realm of formal synthesis of HSs, different methods [20, 41] can be classified along several dimensions. (i) Along the designable part of the system, the synthesis problem can be categorized into *feedback controller* synthesis [37], *switching controller* synthesis [19, 21], and *reset controller* synthesis [8, 24, 40].

(ii) Along the properties of interest, the problem can be classified into *safety* controller synthesis, *liveness* controller synthesis, etc.

Switching controller synthesis [21], shaping HSs by strategically constraining their discrete behavior, can be categorized into two fundamentally approaches. The first is based on constraint solving [41,54]. This approach highly depends on finding suitable certificate templates, which is challenging to generate manually. The other approach is abstraction-based method. Given its capability to easily handle complex temporal specifications, this method has been increasingly adopted in recent research [4,16,25].

The synthesis of HSs concerning reach-avoid type specifications, similar to those discussed in this paper, predominantly focuses on feedback controllers. Notable methods include the Counterexample-Guided Inductive Synthesis (CEGIS) approach proposed by Hadi and Sriram [35,36], optimization-based methods [51], and others [15,31].

When considering STL as the specification, most works have focused solely on the continuous dynamics of HSs. Raman et al. proposed a method to encode the STL specification of a hybrid system into Mixed Integer Linear Programming (MILP) [33]. This method was employed to synthesize a robust controller in a CEGIS manner in [34]. Synthesizing a controller by reinforcement learning technique for an essential discrete-time system is also introduced recently [28]. The Control Barrier Function-based method can also be used to synthesize feedback controller with respect to STL, without requiring discretization of the continuous system [22]. While [38] is the only work we know that considers synthesizing switched systems with respect to STL specification, the synthesized part is the switch input for the hybrid automata with discrete dynamics. In contrast, our work is aimed at synthesizing switching controllers that determine the switch time for the system.

Although numerous studies [35,36] address the reach-avoid type specifications of hybrid systems discussed in this paper, the majority of them focus on feedback controllers rather than switching controllers.

8 Conclusion

We proposed a novel method to synthesize switching controllers for HSs against a fragment of the STL. Our method iteratively calculates the state-time set for each mode, which services as foundation of the synthesize algorithm. The distinctive feature of our approach lies in its soundness and relative completeness. Our preliminary experiments, leveraging a range of notable examples from existing literature, have effectively demonstrated the method's efficiency and efficacy.

For future work, we plan to continue to explore in two directions. (i) Enlarge the range of specifications under consideration to encompass general STL formulas featuring nested temporal operators. The primary challenge here is devising a unified, recursive formula reasoning approach for general STL specifications. (ii) Broaden the types of controllers that can be synthesized from the calculated state-time sets.

References

1. Alur, R., Dill, D.L.: A theory of timed automata. *Theoretical computer science* **126**(2), 183–235 (1994)
2. Arnon, D.S., Collins, G.E., McCallum, S.: Cylindrical algebraic decomposition i: The basic algorithm. *SIAM Journal on Computing* **13**(4), 865–877 (1984)
3. Atkins, E.M., Bradley, J.M.: Aerospace cyber-physical systems education. In: *AIAA Infotech@ Aerospace (I@A) Conference*. p. 4809 (2013)
4. Aydin Gol, E., Lazar, M., Belta, C.: Language-guided controller synthesis for discrete-time linear systems. In: *Proceedings of the 15th ACM international conference on Hybrid Systems: Computation and Control*. pp. 95–104 (2012)
5. Bae, K., Lee, J.: Bounded model checking of signal temporal logic properties using syntactic separation. *Proceedings of the ACM on Programming Languages* **3**(POPL), 1–30 (2019)
6. Chen, X., Abraham, E., Sankaranarayanan, S.: Taylor model flowpipe construction for non-linear hybrid systems. In: *2012 IEEE 33rd Real-Time Systems Symposium*. pp. 183–192. IEEE (2012)
7. Chen, X., Abraham, E., Sankaranarayanan, S.: Flow*: An analyzer for non-linear hybrid systems. In: *Computer Aided Verification: 25th International Conference, CAV 2013, Saint Petersburg, Russia, July 13–19, 2013. Proceedings 25*. pp. 258–263. Springer (2013)
8. Clegg, J.C.: A nonlinear integrator for servomechanisms. *Transactions of the American Institute of Electrical Engineers, Part II: Applications and Industry* **77**(1), 41–42 (1958)
9. De Alfaro, L., Faella, M., Henzinger, T.A., Majumdar, R., Stoelinga, M.: The element of surprise in timed games. In: *International Conference on Concurrency Theory*. pp. 144–158. Springer (2003)
10. De Moura, L., Bjørner, N.: Z3: An efficient smt solver. In: *International conference on Tools and Algorithms for the Construction and Analysis of Systems*. pp. 337–340. Springer (2008)
11. Deshmukh, J.V., Sankaranarayanan, S.: Formal techniques for verification and testing of cyber-physical systems. *Design Automation of Cyber-Physical Systems* pp. 69–105 (2019)
12. Engell, S., Kowalewski, S., Schulz, C., Stursberg, O.: Continuous-discrete interactions in chemical processing plants. *Proceedings of the IEEE* **88**(7), 1050–1068 (2000)
13. Feng, S., Chen, M., Xue, B., Sankaranarayanan, S., Zhan, N.: Unbounded-time safety verification of stochastic differential dynamics. In: Lahiri, S.K., Wang, C. (eds.) *Computer Aided Verification - 32nd International Conference, CAV 2020, Los Angeles, CA, USA, July 21–24, 2020, Proceedings, Part II. Lecture Notes in Computer Science*, vol. 12225, pp. 327–348. Springer (2020)
14. Feng, S., Chen, M., Zhan, N., Fränzle, M., Xue, B.: Taming delays in dynamical systems - unbounded verification of delay differential equations. In: Dillig, I., Tasiran, S. (eds.) *Computer Aided Verification - 31st International Conference, CAV 2019, New York City, NY, USA, July 15–18, 2019, Proceedings, Part I. Lecture Notes in Computer Science*, vol. 11561, pp. 650–669. Springer (2019)
15. Fränzle, M., Chen, M., Kröger, P.: In memory of oded maler: automatic reachability analysis of hybrid-state automata. *ACM SIGLOG News* **6**(1), 19–39 (2019)
16. Girard, A.: Controller synthesis for safety and reachability via approximate bisimulation. *Automatica* **48**(5), 947–953 (2012)

17. Henzinger, T.A.: The theory of hybrid automata. In: Proceedings 11th Annual IEEE Symposium on Logic in Computer Science. pp. 278–292. IEEE (1996)
18. Henzinger, T.A., Majumdar, R.: Symbolic model checking for rectangular hybrid systems. In: International Conference on Tools and Algorithms for the Construction and Analysis of Systems. pp. 142–156. Springer (2000)
19. Jha, S., Seshia, S.A., Tiwari, A.: Synthesis of optimal switching logic for hybrid systems. In: Proceedings of the ninth ACM international conference on Embedded software. pp. 107–116 (2011)
20. Jin, X., An, J., Zhan, B., Zhan, N., Zhang, M.: Inferring switched nonlinear dynamical systems. *Formal Aspects of Computing* **33**(3), 385–406 (2021)
21. Liberzon, D.: *Switching in systems and control*, vol. 190. Springer (2003)
22. Lindemann, L., Dimarogonas, D.V.: Control barrier functions for signal temporal logic tasks. *IEEE control systems letters* **3**(1), 96–101 (2018)
23. Lindemann, L., Nowak, J., Schönbächler, L., Guo, M., Tumova, J., Dimarogonas, D.V.: Coupled multi-robot systems under linear temporal logic and signal temporal logic tasks. *IEEE Transactions on Control Systems Technology* **29**(2), 858–865 (2019)
24. Liu, J., Su, H., Bai, Y., Gu, B., Xue, B., Yang, M., Zhan, N.: Correct-by-construction for hybrid systems by synthesizing reset controller. *arXiv preprint arXiv:2309.05906* (2023)
25. Liu, J., Ozay, N., Topcu, U., Murray, R.M.: Synthesis of reactive switching protocols from temporal logic specifications. *IEEE Transactions on Automatic Control* **58**(7), 1771–1785 (2013)
26. Maler, O., Nickovic, D.: Monitoring temporal properties of continuous signals. In: International Symposium on Formal Techniques in Real-Time and Fault-Tolerant Systems. pp. 152–166. Springer (2004)
27. Mazo Jr, M., Davitian, A., Tabuada, P.: Pessoa: A tool for embedded controller synthesis. In: International conference on computer aided verification. pp. 566–569. Springer (2010)
28. Meng, Y., Fan, C.: Signal temporal logic neural predictive control. *IEEE Robotics and Automation Letters* (2023)
29. Mosek, A.: The MOSEK optimization toolbox for MATLAB manual. version 7.1 (revision 28). <http://mosek.com>, (accessed on March 20, 2015) (2015)
30. Prajna, S., Jadbabaie, A., Pappas, G.J.: A framework for worst-case and stochastic safety verification using barrier certificates. *IEEE Transactions on Automatic Control* **52**(8), 1415–1428 (2007)
31. Prajna, S., Rantzer, A.: Convex programs for temporal verification of nonlinear dynamical systems. *SIAM Journal on Control and Optimization* **46**(3), 999–1021 (2007)
32. Raisch, J., Klein, E., Meder, C., Itigin, A., O’Young, S.: Approximating automata and discrete control for continuous systems—two examples from process control. In: *Hybrid Systems V* 5. pp. 279–303. Springer (1999)
33. Raman, V., Donzé, A., Maasoumy, M., Murray, R.M., Sangiovanni-Vincentelli, A., Seshia, S.A.: Model predictive control with signal temporal logic specifications. In: 53rd IEEE Conference on Decision and Control. pp. 81–87. IEEE (2014)
34. Raman, V., Donzé, A., Sadigh, D., Murray, R.M., Seshia, S.A.: Reactive synthesis from signal temporal logic specifications. In: Proceedings of the 18th international conference on hybrid systems: Computation and control. pp. 239–248 (2015)
35. Ravanbakhsh, H., Sankaranarayanan, S.: Counterexample-guided stabilization of switched systems using control lyapunov functions. In: Proceedings of the 18th

- International Conference on Hybrid Systems: Computation and Control. pp. 297–298 (2015)
36. Ravanbakhsh, H., Sankaranarayanan, S.: Robust controller synthesis of switched systems using counterexample guided framework. In: Proceedings of the 13th International Conference on Embedded Software. pp. 1–10 (2016)
 37. Sanfelice, R.G.: Hybrid feedback control. Princeton University Press (2021)
 38. da Silva, R.R., Kurtz, V., Lin, H.: Symbolic control of hybrid systems from signal temporal logic specifications. *Guidance, Navigation and Control* **1**(02), 2150008 (2021)
 39. Su, H., Feng, S., Zhan, S., Zhan, N.: Switching controller synthesis for hybrid systems against stl formulas. *arXiv preprint arXiv:2406.16588* (2024)
 40. Su, H., Zhu, J., Feng, S., Bai, Y., Gu, B., Liu, J., Yang, M., Zhan, N.: Reset controller synthesis by reach-avoid analysis for delay hybrid systems. *arXiv preprint arXiv:2309.05908* (2023)
 41. Taly, A., Gulwani, S., Tiwari, A.: Synthesizing switching logic using constraint solving. *International journal on software tools for technology transfer* **13**(6), 519–535 (2011)
 42. Tomlin, C.J., Lygeros, J., Sastry, S.S.: A game theoretic approach to controller design for hybrid systems. *Proceedings of the IEEE* **88**(7), 949–970 (2000)
 43. Wang, Y., Zhan, S., Wang, Z., Huang, C., Wang, Z., Yang, Z., Zhu, Q.: Joint differentiable optimization and verification for certified reinforcement learning. In: Proceedings of the ACM/IEEE 14th International Conference on Cyber-Physical Systems (with CPS-IoT Week 2023). pp. 132–141 (2023)
 44. Wang, Y., Zhan, S.S., Jiao, R., Wang, Z., Jin, W., Yang, Z., Wang, Z., Huang, C., Zhu, Q.: Enforcing hard constraints with soft barriers: Safe reinforcement learning in unknown stochastic environments. In: International Conference on Machine Learning. pp. 36593–36604. PMLR (2023)
 45. Weispfenning, V.: The complexity of linear problems in fields. *Journal of symbolic computation* **5**(1-2), 3–27 (1988)
 46. Witsenhausen, H.: A class of hybrid-state continuous-time dynamic systems. *IEEE Transactions on Automatic Control* **11**(2), 161–167 (1966)
 47. Wu, Q., Zhan, S.S., Wang, Y., Lin, C.W., Lv, C., Zhu, Q., Huang, C.: Boosting long-delayed reinforcement learning with auxiliary short-delayed task. *arXiv preprint arXiv:2402.03141* (2024)
 48. Wu, Q., Zhan, S.S., Wang, Y., Wang, Y., Lin, C.W., Lv, C., Zhu, Q., Huang, C.: Variational delayed policy optimization. *arXiv preprint arXiv:2405.14226* (2024)
 49. Xue, B., Fränzle, M., Zhan, N.: Inner-approximating reachable sets for polynomial systems with time-varying uncertainties. *IEEE Transactions on Automatic Control* **65**(4), 1468–1483 (2019)
 50. Xue, B., She, Z., Easwaran, A.: Under-approximating backward reachable sets by polytopes. In: Computer Aided Verification: 28th International Conference, CAV 2016, Toronto, ON, Canada, July 17–23, 2016, Proceedings, Part I 28. pp. 457–476. Springer (2016)
 51. Xue, B., Zhan, N., Fränzle, M., Wang, J., Liu, W.: Reach-avoid verification based on convex optimization. *IEEE Transactions on Automatic Control* (2023)
 52. Ye, P., Entcheva, E., Smolka, S.A., Grosu, R.: Modelling excitable cells using cycle-linear hybrid automata. *IET systems biology* **2**(1), 24–32 (2008)
 53. Zhan, S.S., Wang, Y., Wu, Q., Jiao, R., Huang, C., Zhu, Q.: State-wise safe reinforcement learning with pixel observations. *arXiv preprint arXiv:2311.02227* (2023)

54. Zhao, H., Zhan, N., Kapur, D.: Synthesizing switching controllers for hybrid systems by generating invariants. *Theories of Programming and Formal Methods: Essays Dedicated to Jifeng He on the Occasion of His 70th Birthday* pp. 354–373 (2013)
55. Zou, L., Fränzle, M., Zhan, N., Mosaad, P.N.: Automatic verification of stability and safety for delay differential equations. In: *International Conference on Computer Aided Verification*. pp. 338–355. Springer (2015)